

Auto-encoders & Latent Space Visualization

Idant Srivastava

February, 2026

Contents

1	The Standard Autoencoder	2
1.1	Encoder	2
1.2	Bottleneck	2
1.3	Decoder	2
1.4	Limitations of Standard Autoencoders	2
2	The Variational Autoencoder (VAE)	3
3	Latent Variable Generative Modeling	3
3.1	Variational Inference	3
3.2	Evidence Lower Bound (ELBO)	4
3.3	Reparameterization Trick	4
4	Visualization Comparison Guide	5
4.1	PCA, t-SNE, and UMAP: Comparison Table	5
4.2	Choosing Between PCA, t-SNE, and UMAP	5
5	Parameter Analysis	6
5.1	t-SNE: Effect of Perplexity	6
5.2	UMAP: Effect of <code>n_neighbors</code>	6

1 The Standard Autoencoder

Autoencoders are a special type of neural networks that learn to compress data into a compact form and then reconstruct it to closely match the original input. They consist of an encoder, a bottleneck and a decoder.

1.1 Encoder

It compress the input data into a smaller, more manageable form by reducing its dimensionality while preserving important information. It has three layers: input, hidden, and output

- The input layer is where raw data is entered.
- The hidden layers applies weights and activations which compress the data.
- The encoder outputs a compressed vector known as the latent representation or encoding.

1.2 Bottleneck

It is the smallest layer of the network which represents the most compressed version of the input data. It serves as the information bottleneck which force the network to prioritize the most significant features. This helps the model learn the structure and key patterns of the input helps in enabling better generalization and efficient data encoding.

1.3 Decoder

Decoder tries to best restore the the original data. The reconstructed output is then compared to the original encoder to measure the efficacy of the autoencoder. The difference between the output and ground truth is called reconstruction error.

AE Loss Function

$$\begin{aligned}z &= f_{\theta}(x) \\ \hat{x} &= g_{\phi}(z) \\ \hat{x} &= g_{\phi}(f_{\theta}(x)) \\ L(x, \hat{x}) &= \|x - \hat{x}\|^2\end{aligned}$$

1.4 Limitations of Standard Autoencoders

Although autoencoders compress data effectively, they do not function well as generative models. The latent space learned by an AE is often irregular and discontinuous.

Since the encoder maps each input to a single point z , the latent space may contain regions that were never observed during training. Sampling random latent vectors from such regions and passing them through the decoder typically produces meaningless outputs.

Therefore, standard autoencoders lack a structured latent distribution that allows reliable generation of new samples.

2 The Variational Autoencoder (VAE)

Variational Autoencoders (VAEs) are generative models that learn the probability distribution of data so that they can generate new samples similar to the training data. They combine ideas from autoencoders, probability theory, and variational inference.

3 Latent Variable Generative Modeling

To build a generative model, we assume that data is generated from a latent variable z . The generative process is defined as:

$$z \sim p(z)$$

$$x \sim p_\theta(x|z)$$

The probability of observing data x is obtained by marginalizing over z :

$$p_\theta(x) = \int p_\theta(x|z)p(z)dz$$

The objective is to maximize the data likelihood:

$$\max_{\theta} \log p_\theta(x)$$

However, this integral is generally intractable.

3.1 Variational Inference

To address this problem, we introduce an approximate posterior distribution:

$$q_\phi(z|x)$$

which approximates the true posterior $p(z|x)$.

In a Variational Autoencoder, the encoder predicts parameters of this distribution. A common choice is a Gaussian distribution:

$$q_\phi(z|x) = \mathcal{N}(\mu(x), \sigma(x)^2)$$

Instead of mapping inputs to deterministic points, the encoder now maps them to probability distributions.

3.2 Evidence Lower Bound (ELBO)

We begin with the log likelihood:

$$\log p_\theta(x)$$

Using the variational distribution:

$$\log p_\theta(x) = \log \int q_\phi(z|x) \frac{p_\theta(x, z)}{q_\phi(z|x)} dz$$

Applying Jensen's inequality yields the Evidence Lower Bound (ELBO):

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right]$$

Since

$$p_\theta(x, z) = p_\theta(x|z)p(z)$$

the ELBO becomes:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p(z))$$

This objective consists of two components:

- Reconstruction term: ensures the decoder reconstructs the input accurately.
- KL divergence term: forces the latent distribution to stay close to the prior $p(z)$.

3.3 Reparameterization Trick

Sampling directly from $q_\phi(z|x)$ prevents gradient-based optimization. To allow backpropagation, the sampling step is rewritten as:

$$z = \mu(x) + \epsilon\sigma(x)$$

where

$$\epsilon \sim \mathcal{N}(0, 1)$$

This transformation separates the randomness from the learnable parameters μ and σ , enabling gradients to propagate through the network.

4 Visualization Comparison Guide

4.1 PCA, t-SNE, and UMAP: Comparison Table

Property	PCA	t-SNE	UMAP
Linearity	Linear	Non-linear	Non-linear
Global vs. Local	Global structure	Local structure	Both (local priority)
Computational Cost	very fast	slow for $n > 10^4$	much faster
Primary Use Case	Preprocessing, noise reduction, feature extraction	2D/3D cluster visualization	Scalable visualization, manifold learning

4.2 Choosing Between PCA, t-SNE, and UMAP

1. Check whether the relationship between features is linear: If the relationships among features are linear then use **PCA**.

If the data contains nonlinear relationships then PCA may fail to capture meaningful structure, and nonlinear methods such as t-SNE or UMAP should be considered.

2. Consider the dataset size: If the number of data points is very large ($n > 50,000$), it is generally recommended to use **UMAP**.

- t-SNE becomes computationally expensive for very large datasets.
- UMAP is designed to scale efficiently and can handle large datasets much faster.

If the dataset is smaller then both t-SNE and UMAP are good options.

3. Determine whether global structure or trajectories are important: If the goal is to preserve global relationships or continuous trajectories in the data, then **UMAP** is the better choice.

- UMAP preserves more global structure of the data.
- It performs well when the data contains smooth transitions or developmental trajectories.

If the primary goal is to achieve the **clearest possible cluster separation**, then **t-SNE** is often preferred.

- t-SNE strongly emphasizes local neighborhood structure.
- It typically produces very distinct clusters in visualization.

5 Parameter Analysis

5.1 t-SNE: Effect of Perplexity

Perplexity P controls the effective number of neighbours each point considers. The bandwidth σ_i for each point is chosen so that the entropy of its neighbour distribution equals $\log_2 P$:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2/2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2/2\sigma_i^2)}$$

Perplexity	Structure captured	Effect on embedding
2–5	Micro-local only	Clusters split into many fragments; noise amplified
30–50	Balanced	Well-separated clusters; recommended default
100–200	Near-global	Clusters merge; borders blur
$\geq n/3$	Global	Structure collapses; near-uniform spread

5.2 UMAP: Effect of `n_neighbors`

`n_neighbors` controls how many nearest neighbours each point connects to when building the graph. The edge weight decays with distance:

$$w(x_i, x_j) = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

where ρ_i is the distance to the nearest neighbour and σ_i normalizes the weights.

<code>n_neighbors</code>	Graph character	Effect on embedding
15–30	Balanced	Compact clusters; recommended default (UMAP default is 15)
50–100	More global	Clusters compress; trajectories become visible
200+	Near-global	Cluster identity softens; global topology dominates

Increasing `n_neighbors` trades local detail for global structure in a smooth, continuous way, making UMAP easier to interpret across parameter values.